

R을 활용한 수치 알고리즘 구현

R 프로그래밍 with workflow design



수치 알고리즘의 이해

R을 활용한 수치 알고리즘 구현

- **알고리즘**은 '문제 해결 절차'를 기계가 이해 및 실행할 수 있도록 **수학적 사고**와 **프로그래밍**을 연결하는 핵심 도구임
- 많은 이들이 R 문법을 익혀도 **무엇을**·**어떻게** 구현할지 막히는 이유는 수식을 알고리즘으로 변환하고 다시 코딩으로 전환하는 사고의 틀이 부족하기 때문임
- 단순히 언어 문법을 아는 것을 넘어 수학적 모델을 코드로 실현하는 능력이 진정한 프로그래머의 자질임
- 수학적 아이디어를 코드로 옮길 수 있는 힘이야말로 데이터 과학자·통계학자가 반드시 갖춰야 할 역량임

알고리즘 구현을 위한 프로그래밍 가이드라인

문제 이해와 전략 탐색

1. 해결하고자 하는 문제를 명확하게 이해하는 과정
    - 입출력 정의: 어떤 데이터가 들어오고, 어떤 결과를 내야하는지 파악
    - 수학적 배경: 알고리즘이 해결해야하는 수학적 문제 (예: 선형 방정식, 최적화 등) 파악
2. 문제 해결을 위한 방법을 도출하거나 검색하는 과정
    - 가능한 알고리즘 리스트업(가우스 소거 vs. 역행렬 활용 등)
    - 기존의 여러 알고리즘을 응용 및 새롭게 조합
    - 문제를 기능에 따른 모듈(module) 단위로 검토

알고리즘 구현과 검증

3. 프로그래밍 언어로 구현할 수 있도록 방법을 구체화하는 과정
    - 전체 알고리즘을 세분화하고 각 부분을 하나씩 구현
4. 프로그래밍 언어로 구현하는 과정
5. 구현된 프로그램을 확인하는 과정
    - 여러 예제를 활용해 구현 결과를 검증
    - 예외적인 상황에 대처가 잘되는지 확인

정렬 알고리즘을 통한 구현 과정 이해

정렬 알고리즘으로 배우는 구현 흐름

- 목표: 유한한 길이의 수열을 오름차순으로 정렬하는 프로그램 작성
- 구현 단계
    1. 문제를 명확하게 이해하는 과정
    2. 문제 해결을 위한 방법을 도출하거나 검색하는 과정
    3. 프로그래밍 언어로 구현할 수 있도록 방법을 구체화하는 과정
    4. 프로그래밍 언어로 구현하는 과정
    5. 구현된 프로그램을 확인하는 과정

1. 문제를 명확하게 이해하는 과정

- 해결해야 할 문제: 숫자 벡터를 오름차순으로 정렬하는 것
- 예: 입력 `(6, 9, 2, 12, 7, 4)` → 출력 `(2, 4, 6, 7, 9, 12)`

2. 문제 해결을 위한 방법을 도출하거나 검색하는 과정

- 문제 해결을 위해 알고리즘을 고안하거나 참고 자료를 검색할 수 있음
- 여기서는 **선택 정렬(**selection sort**)** 알고리즘을 사용
- 선택 정렬 방식
    1) 배열에서 가장 작은 값을 찾아 맨 앞과 교환
    2) 그다음 작은 값을 두 번째 위치와 교환
    3) 이 과정을 끝까지 반복하며 정렬 수행

3. 프로그래밍 언어로 구현할 수 있도록 방법을 구체화하는 과정

- 알고리즘을 실제로 구현하려면 언어 친화적으로 구체화하는 과정이 필요함
- 선택 정렬은 반복문을 사용해 벡터의 각 위치마다 가장 작은 값을 찾아 교환하는 방식
- 핵심 단계
  - 현재 위치부터 끝까지 최솟값의 인덱스를 탐색
  - 해당 인덱스와 현재 위치의 값을 **임시 변수**를 통해 교환

3. 프로그래밍 언어로 구현할 수 있도록 방법을 구체화하는 과정: 의사 코드

```
#  첫 번째 위치부터 마지막 위치까지 반복
for i in 1 to n-1
   #  최솟값의 인덱스 초기화
  min_index = i
   #  i번째 이후에 원소들과 크기 비교
  for j in (i+1) to n
        if  j번째 숫자 < min_index에 해당하는 숫자
       #  min_index를 더 작은 값의 위치로 업데이트
  #  min_index가 업데이트 되었는지 확인
  #  (업데이트 되지 않았다면 이미 i번째 위치 숫자가 i번째로 작은 숫자)
   if min_index != i
      #  임시 변수 생성 후 min_index에 해당하는 숫자와 i번째 숫자 교환
    temp = i번째 숫자
    i번째 숫자 = min_index에 해당하는 숫자
    min_index에 해당하는 숫자 = temp
retun 정렬된 벡터
```

4. 프로그래밍 언어로 구현하는 과정

- 상위 과정을 잘 설계했다면 R로의 구현은 단순
- 재사용성을 위해 함수 형태로 작성하는 것이 좋음
- R 문법은 구조 구현이 어려울 때만 중요하며 대부분은 알고리즘 설계가 핵심

4. 프로그래밍 언어로 구현하는 과정: R 코드

```r
selection_sort = function(vec) {
  n = length(vec)  # 길이가 n인 입력 벡터
  for (i in 1:(n-1))
  {
    min_index = i
    for (j in (i+1):n)
    {
      if (vec[j] < vec[min_index])
      {
        min_index = j
      }
    }
    if (min_index != i)
    {
      temp = vec[i]
      vec[i] = vec[min_index]
      vec[min_index] = temp
    }
  }
  return(vec)
}
```

5. 구현된 프로그램을 확인하는 과정

- 구현한 코드는 하나의 예제가 아닌 여러 예제를 통해 검증
- 예1: 상수 벡터 입력

```
vec = c(6, 9, 2, 12, 7, 4)
sorted_vec = selection_sort(vec)
print(sorted_vec)
```

```
## [1]  2  4  6  7  9 12
```

- 예2: 랜덤 벡터 입력

```
set.seed(1234)
vec = runif(5)
print(vec)
```

```
## [1] 0.1137034 0.6222994 0.6092747 0.6233794 0.8609154
```

```
print(selection_sort(vec))
```

```
## [1] 0.1137034 0.6092747 0.6222994 0.6233794 0.8609154
```

5. 구현된 프로그램을 확인하는 과정: 예외 처리

- 예3: 실수 입력

```
vec = 3
sorted_vec = selection_sort(vec)
```

```
Error in if (vec[j] < vec[min_index]) { :
  missing value where TRUE/FALSE needed
```

- 예외 처리

```
selection_sort = function(vec) {
  n = length(vec)  # 길이가 n인 입력 벡터
  if (n < 2) return(vec)   # 입력 벡터 길이가 1인 경우

  ...
}
```

```
vec = 3
print(selection_sort(vec))
```

```
## [1] 3
```

수치 선형 대수: 가우스 소거법

가우스 소거법으로 배우는 구현 흐름

- 목표: 연립선형방정식 $Ax = b$의 해 x를 구하기 위한 기본 알고리즘인 **가우스 소거법 (*Gaussian elimination*)** 구현
- 절차
  - 전진 소거 (Forward Elimination)
    - 행 연산을 통해 계수 행렬 A를 상삼각 행렬로 변환
    - 아래 행의 항들을 피벗(pivot)을 기준으로 소거
  - 후진 대입 (Back Substitution)
    - 상삼각 행렬로부터 마지막 행부터 거꾸로 해를 계산
- 복잡한 연립방정식을 단계적으로 간단한 형태로 바꾸어 해를 구하는 직관적이고 효율적인 기법

문제를 명확하게 이해하는 과정

- 목표: 연립선형방정식 $Ax = b$에서 미지수 벡터 x를 구함
  - A: $n \times n$ 계수 행렬 (coefficient matrix)
  - x: $n \times 1$ 미지수 벡터 (unknown vector)
  - b: $n \times 1$ 상수 벡터 (constant vector)
- 예제

$$\begin{cases} 3x_1 + x_2 + 2x_3 = 5 \\ 2x_1 - 3x_2 + 4x_3 = -3 \\ x_1 + 2x_2 - 2x_3 = 6 \end{cases}$$

- 행렬 형태:

$$A = \begin{bmatrix} 3 & 1 & 2 \\ 2 & -3 & 4 \\ 1 & 2 & -2 \end{bmatrix}, \quad b = \begin{bmatrix} 5 \\ -3 \\ 6 \end{bmatrix}$$

문제 해결을 위한 방법을 도출하거나 검색하는 과정

1. 전진 소거:

- 첫 번째 열의 피벗(Pivot) 원소를 기준으로 아래 행의 요소들을 소거
- 두 번째 열에서도 동일한 과정을 반복하여 상삼각행렬 형태로 변환한다.

2. 후진대입:

- 상삼각행렬이 만들어 지면, 마지막 행부터 역순으로 대입하여 x 값을 구한다.

전진 소거 (Forward Elimination)의 수식적 이해

- 핵심 연산
  - 소거 계수 (elimination factor) 계산

$$m_{ik} = \frac{A_{ik}}{A_{kk}}$$

  - 행 갱신

$$A_{ij} \leftarrow A_{ij} - m_{ik} \cdot A_{kj} \quad (\forall j \geq k)$$

  - 상수 벡터도 함께 갱신

$$b_i \leftarrow b_i - m_{ik} \cdot b_k$$

후진 대입 (Back Substitution)의 수식적 이해

- 역방향 해 구하기
  - 마지막 식

$$x_n = \frac{b'_n}{A_{nn}}$$

  - 그 위부터는 다음과 같이 계산

$$x_i = \frac{b'_n - \sum_{j=i+1}^{n} A_{ij} x_j}{A_{ii}}$$

- [주의] $A_{ii} \neq 0$이어야 함 (단수 행렬 예외 처리 필요)

알고리즘을 코드로 구현하기 위한 틀

```
# 1. 전진 소거
for k in 1 to (n - 1): # k번째 열을 피벗으로 설정
  for i in (k + 1) t on: # k번째 열 아래의 행을 소거
    factor = A[i, k] / A[k, k] # 소거 계수 계산
    for j in k to n: # k번째 열 이후 모든 열 업데이트
      A[i, j] = A[i, j] - factor * A[k, j]
        b[i] = b[i] - factor * b[k] # 상수 벡터 b도 같이 업데이트

# 2. 후진 대입
x = 0으로 채워진 길이 n 벡터
for i in n down to 1: # 마지막 행부터 역순으로 대입
  sum_ax = 0
for j in (i + 1) to n:
    sum_ax = sum_ax + A[i, j] * x[j] # 이미 구한 변수 활용
x[i] = (b[i] - sum_ax) / A[i, i] # 최종 x값 계산

return x # 해 벡터 반환
```

프로그래밍 언어로 구현하는 과정

```
gaussian_elimination = function(A, b)
{
  n = nrow(A)


  for (k in 1:(n - 1)){ # 전진 대입
    for (i in (k + 1):n){
      factor = A[i, k] / A[k, k]
      A[i, k:n] = A[i, k:n] - factor * A[k, k:n]
      b[i] = b[i] - factor * b[k]
    }
  }

  x = numeric(n) # 후진 대입
  for (i in n:1){
    if (i < n)
      sum_ax = sum(A[i, (i+1):n] * x[(i+1):n])
    else
      sum_ax = 0   ## 마지막 변수일 경우 sum_ax는 0

    x[i] = (b[i] - sum_ax) / A[i, i]
  }
```

구현된 프로그램을 확인하는 과정

- 예제 실행으로 검증

```
A = matrix(c(3, 1, 2, 2, -3, 4, 1, 2, -2), nrow=3, byrow=TRUE)
b = c(5, -3, 6)
solution = gaussian_elimination(A, b)
print(solution)
```

```
## [1]  2  1 -1
```

- 해 $x = (2, 1, -1)$: 원래 선형 방정식 $Ax = b$ 만족
- 예외 상황 확인
  - 특이(singular) 행렬일 경우 $A_{kk} = 0$이 되므로 나눗셈 오류 발생 가능
  - 적절한 `stop()` 처리 필요
- 다양한 크기의 행렬에 대해 검증해야 안정성 확보 가능